

Evan Baesler
September 20, 2026
31151619

EE Design 1 Technical Report

Microcontroller Basics

Introduction

In this project, we are to design a Raspberry Pico 2 system to strobe an LED at 2Hz, or utilize a piezo buzzer at 2kHz, independently shutting off the other peripheral so that only one is utilized at a time; controlling the mode with a debounced push-button via interrupt.

Methods

To create this schematic, we utilized KiCAD to connect our systems to individual IO ports; utilizing GPIO0,3,12. GPIO0/3 would be outputs, and GPIO12 would be an input for our button.

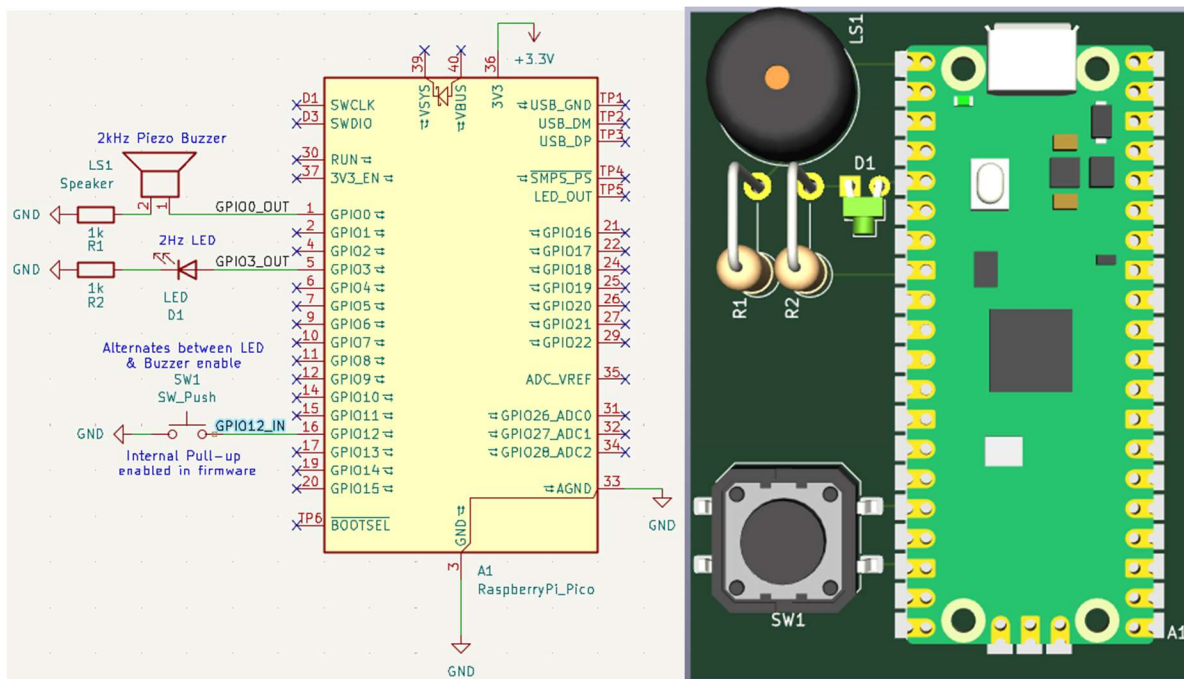


Figure 1: KiCAD Schematic of the Raspberry Pico 2 Buzzer, LED, and Button

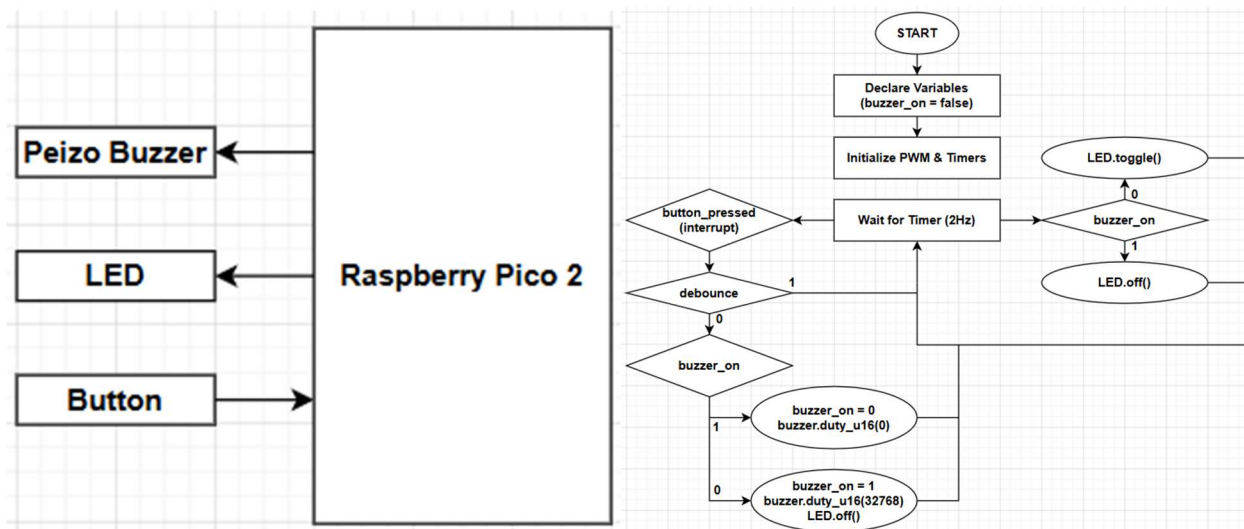


Figure 2: Hardware Flowchart (Left), Program Flowchart (Right)

For our program flowchart, we start by declaring our variables and initializing relevant modules (2Hz timer for the LED, 2kHz PWM for the buzzer.) When the timer ends, it would then check the buzzer_on variable (which starts as false by default), if it is off, it toggles the LED, if it is on, it disables the LED (as the buzzer is operating.)

Concurrently, if the button is pressed at all, this interrupt pauses the polling and first checks for a debounce to ensure the button is actually being pressed, (rather than allowing microscopic contacts to fluctuate the voltage and misfire.) If it is a bounce, it goes back to waiting, otherwise it checks if the buzzer_on variable is true; if it is it turns it off and sets the PWM duty cycle to 0. If the buzzer_on variable is false, it enables it, sets the PWM duty cycle to 32768 (2^{15} or half duty cycle), and disables the LED. The machine would then return to waiting for the timer event.

Results

The following waveforms are scoped from a Rigol Oscilloscope for our 2Hz output and 2kHz output on a breadboard implementation of the above circuits:



Figure 3: 2Hz LED Signal (Left), 2kHz Buzzer & 1kHz Resistor Signal (Right)

As the 2Hz signal utilizes timers rather than PWM, this timer must have an operation of 250ms rather than 500ms due to it needing to do a full two cycles per period (half high, half low), whereas the PWM function accounts for this.

Our signals are selected via an interrupt, which means that when the button is pressed, the main program code stops, and jumps to our interrupt function; rather than continuously iterate until a conditional is met within the main program code (polling.)

Conclusion

This module had us do many of the things we are familiar with, such as polling, interrupts, and IO; and taught us to do it in a new language (MicroPython.) While it is a higher-level language with a compiler, learning the syntax is fundamental to understanding how different microprocessors and microcontrollers are to be programmed.

References

1. Flowcharts created using apps.diagrams.net
2. Programming assistance from AI (ChatGPT, ClaudeAI)

Appendix

Program Code:

```
from machine import Pin
from machine import Timer
from machine import PWM
import time

LEDtimer = Timer()

pin = Pin

# 28 GPIO, do not use pin 3, 8, 13, 18 on left (GND pins.)

# Buzzer: Pin 1 (GP0) -> 2 kHz operation (less than 1 ms T -> PWM)
# LED: Pin 5 (GP3) -> 2 Hz blinking operation
# Button: Pin 16 (GP12) -> Toggle for buzzer

buzzer = PWM(Pin(0))
buzzer.freq(2000)
buzzer.duty_u16(0) # 16-bit register where  $2^{16} = 65535 = 100\%$ 
                  #  $32768 = 2^{15} = 50\%$  duty cycle

led = pin(3, pin.OUT)

button = pin(12, pin.IN, pin.PULL_UP)

buzzer_on = False
last_press = time.ticks_ms()

def timer_2Hz(timer):

    if not buzzer_on:
        led.toggle()

    else:
        led.off()
```

```

def button_pressed(pin):
    global led_on
    global buzzer_on
    global last_press
    global frequency

    now = time.ticks_ms()

    if time.ticks_diff(now, last_press) < 200:
        return

    last_press = now

    if buzzer_on:
        buzzer.duty_u16(0)
        buzzer_on = False
    else:
        buzzer.duty_u16(32768)
        buzzer_on = True
        led.off()

LEDtimer.init(
    period = 250, # 500 ms = 1/2 sec = 2 Hz
    mode = Timer.PERIODIC,
    callback = timer_2Hz
)

button.irq(
    trigger = Pin.IRQ_FALLING,
    handler = button_pressed
)

```